# Design Evolution of a Three-Phase Inverter Prototype for Grid-Following Control

Mark Rifkin and Ian Toyota
ES145 Final Project
May 5, 2026

***Abstract*—This report presents the design progression of a low-voltage inverter prototype developed from ideal LTspice switching models into a breadboarded sinusoidal pulse-width modulation (SPWM) inverter with voltage-sensing and phase-following behavior. The project began with the motivation for inverter-based resources in modern power systems, then used ideal half-bridge and full-bridge models to establish the inverter switching concept. Real MOSFET implementations were then studied, revealing the practical importance of gate-to-source drive voltage, switching delay, dead time, pull-up and pull-down networks, and helper-transistor gate drive behavior. A three-phase half-bridge inverter prototype was developed to convert a DC input into three offset AC outputs. A single-phase full bridge inverter prototype was also developed, with grid-synchronization (to a benchtop reference) implemented via a voltage sense circuit. Both inverter prototypes used sinusoidal PWM logic to generate sinusoidal outputs and RC output filters to isolate the fundamental AC outputs from higher-frequency PWM noise. The full-bridge phase-following result establishes the control structure needed to extend the system toward a three-phase grid-following inverter.**

***Index Terms*—Inverter, SPWM, half bridge, full bridge, MOSFET, STM32, Arduino Nano R4, three-phase inverter, grid-following control development.**

## I. Introduction

Historically, turbine-based generators made up the vast majority of grid generation capacity. Driven by sources like fossil fuels, nuclear fission, and hydro, these generators generate power at the same frequency, voltage, and phase as the voltage on the grid [7]. In the case of a sudden loss of power generation capacity on the grid (e.g., during an extreme weather event), the rotational inertia of the heavy rotors damps the resulting frequency decline. The extra time before the grid drops frequency means generator control systems have more time to increase output and maintain grid stability. As the power grid transitions toward renewable energy, an increasing number of power sources are "inverter-based resources" like solar, wind and batteries, which are tied to the grid through inverters that convert DC power to AC power, rather than directly. For grid-connected operation, the inverter output must be compatible with the voltage magnitude, frequency, and phase. Grid-following inverters are widely used to control the output of AC power from the inverter, acting as a current source that can provide or absorb real and/or reactive power on the grid [6]. However, they cannot act as voltage sources, meaning their only option in case of grid instability is to disconnect. This reduces the amount of conventional inertia on the grid, making it more unstable and vulnerable to outages. Various innovations have been introduced to address this problem, including grid-forming inverters that act as flexible voltage sources on the grid, and can "simulate" inertia by setting the frequency and phase of grid voltage[4]. Large-scale adoption of grid-forming inverters will be essential to enabling a renewable grid [3].

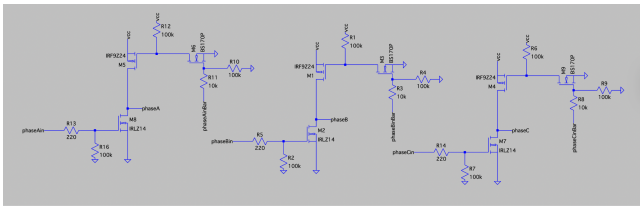

Fig. 1. Three-phase inverter concept implemented as three half bridges sharing the same DC link.

This project aims to build understanding of inverter systems by designing, constructing, and programming prototypes toward a grid-following inverter system. First, the switching hardware and sinusoidal PWM (SPWM) control needed to synthesize AC waveforms from a DC input are developed. The project reached a voltage-following milestone where a function generator was used as a low-voltage grid reference. The measured reference was fed through a voltage-sense path into the controller, and the SPWM phase and frequency followed that reference.

## II. Half-Bridge and Full-Bridge Inverter Concept

A half bridge is the simplest useful inverter switching cell. It places two controlled switches in series across a DC supply and uses their midpoint as the output node. If the high-side switch is on, the output is pulled toward the positive rail; if the low-side switch is on, the output is pulled toward the negative or reference rail. Therefore, a half bridge turns a DC bus into a controlled two-state output.

A full bridge follows naturally from two half bridges. Instead of measuring the load voltage from one bridge midpoint to ground, the load is connected between two bridge midpoints. Driving the two legs with complementary or phase-shifted commands allows the load to see both positive and negative voltage, which is the key reason the full bridge is useful for AC synthesis. The textbook [2] was referenced extensively for these circuits and the later three-phase layout, as well as the theory behind SPWM.

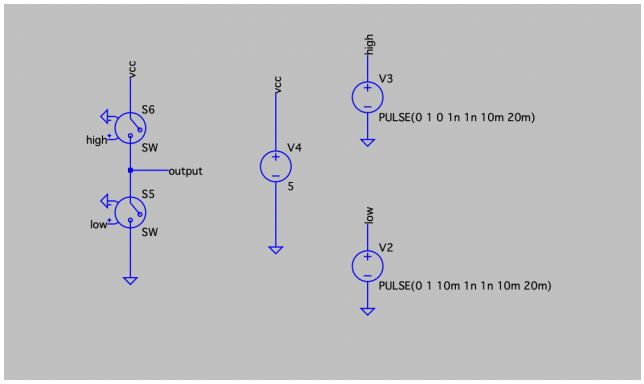


Fig. 2. Ideal half-bridge model using switches rather than physical MOSFETs.

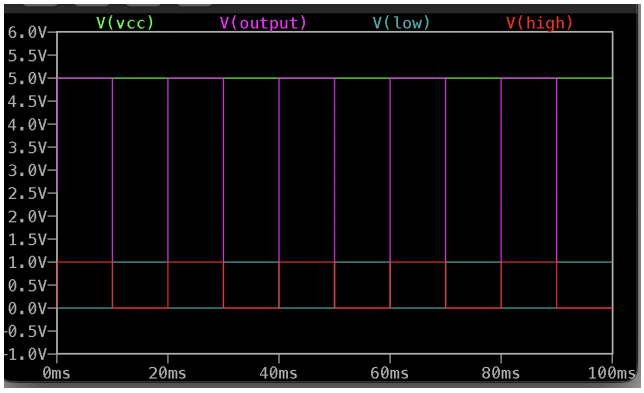


Fig. 3. Ideal half-bridge input-output relationship. The output follows the commanded switching state without transistor drive limitations.

## III. Ideal LTspice Switching Models

The first design stage used ideal switches in LTspice. This was useful because ideal switches isolate the basic inverter concept from the nonideal behavior of real transistors. In the ideal half bridge, the control signal can be treated as an abstract switching command. The switch changes state instantaneously, the required control voltage can be arbitrarily small in the simulation, and the DC bus voltage can be increased without causing gate-drive limitations or visible timing effects.

The ideal model also removes the need for dead-time design. Since the switches are abstract and transition instantly, there is no interval where both devices conduct because one transistor is turning off slowly while the other turns on. This makes the ideal model useful pedagogically, but it hides the major practical design problems that appear in real bridge circuits that we explore below.

The ideal full bridge was used mainly as a conceptual extension. It demonstrated that two half bridges can create a differential output, with the bridge legs driven so that the load voltage changes sign. This result motivated the later transition from half-bridge experiments to a full-bridge and then three-phase bridge structure.

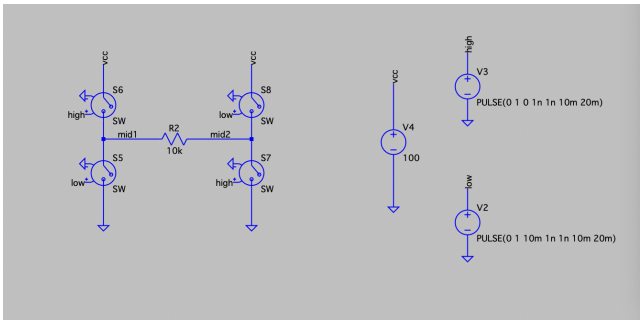


Fig. 4. Ideal full bridge built from two ideal half-bridge legs.

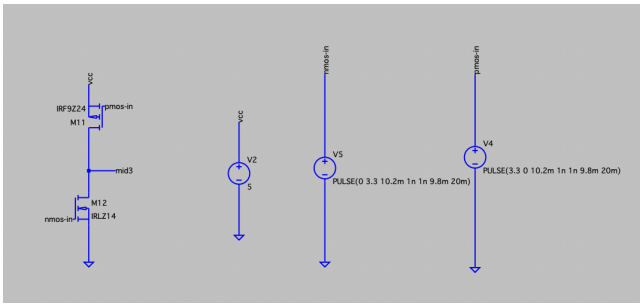


Fig. 5. Primitive MOSFET half bridge using input signals directly and no added dead time or gate network.

## IV. Replacing Ideal Switches With MOSFETs

After the ideal models established the topology, the switches were replaced with actual MOSFET models. The selected devices were the IRF9Z24 PMOS with the following datasheet for the high-side device and the IRLZ14 NMOS (datasheet) for the low-side device. Both were chosen because they are power MOSFETS and would allow a high gate to source (Vgs) and voltage to source (Vds) voltage, which is important for this application.

This replacement changed the behavior drastically. Unlike ideal switches, MOSFETs do not respond only to an abstract input logic level; they respond to the gate-to-source voltage, $v_{GS}$. The high-side and low-side devices therefore, require gate signals referenced correctly to their own source terminals.

As expected, the primitive MOSFET half bridge, whose schematic is shown in Figure 5, did not behave like the ideal-switch circuit. It needed dead time to reduce shoot-through risk, and it needed gate biasing networks so that the transistor gates would not float or switch unpredictably. This was especially important because the design used complementary NMOS and PMOS devices rather than an integrated gate-driver IC.

## V. Gate Network Iteration

The next improvement added a resistor network to the MOSFET gates. The purpose of this network was to give the gates defined pull-up and pull-down paths, improve switching behavior, and reduce floating-node effects. In simulation, the circuit worked better at low bus voltage. At around 5 V, the

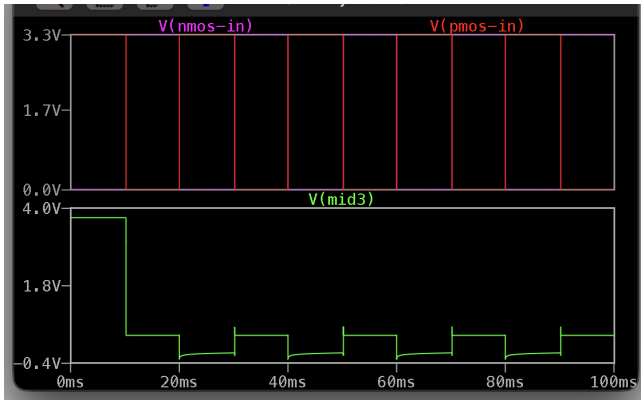


Fig. 6. Nonideal output from the primitive MOSFET half bridge. The circuit no longer behaves like the ideal switch model.

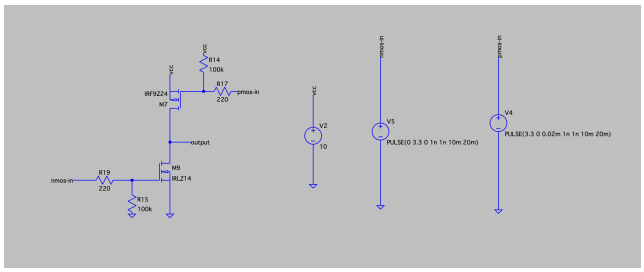

Fig. 7. Half bridge with added gate resistor network.

output waveform followed the expected switching behavior more closely (See Figure 8).

At higher bus voltages, however, the same circuit showed degraded behavior (See Figure 9). The likely cause was a combination of PMOS and NMOS switching mismatch, insufficient gate control, and the lack of a strong gate-driver stage. This demonstrated that a circuit that works at low voltage may fail when the DC bus increases, even if the logical switching command remains unchanged.

The same design was then implemented on the breadboard using a function generator as the input source. The circuit produced an output with the same general shape as the control signal, but the measured output was only approximately 3.4 V. This may have been caused by the circuit itself, but it may also have been limited by using the function generator as the only control source rather than a stronger or more carefully referenced gate-drive stage.

## VI. BS170 Helper-Transistor Stage

To improve the gate-drive behavior, a BS170 helper NMOS transistor was added to the gate of the IRF9Z24 PMOS. The PMOS gate needed to flip between VCC (in the off state) and a sufficiently low voltage in the on state (VG ¡ VCC-VGS threshold). This makes the PMOS impossible to drive with a microcontroller output that is limited to 3.3 or 5V. When enabled with a 3.3 or 5V gate signal, the BS170 pulls the PMOS gate down to ground, and when disabled, the PMOS is pulled up to Vcc.

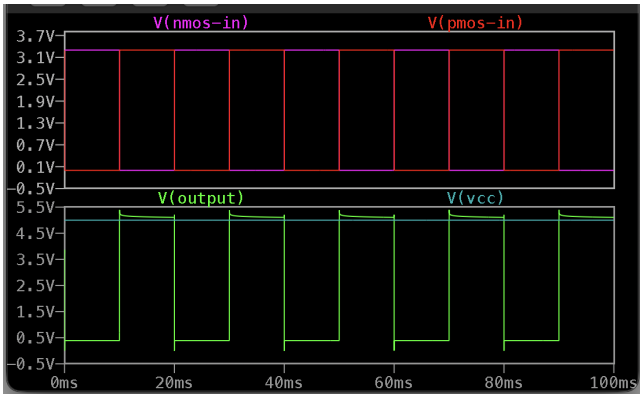


Fig. 8. Low-voltage operation of the resistor-network half bridge.

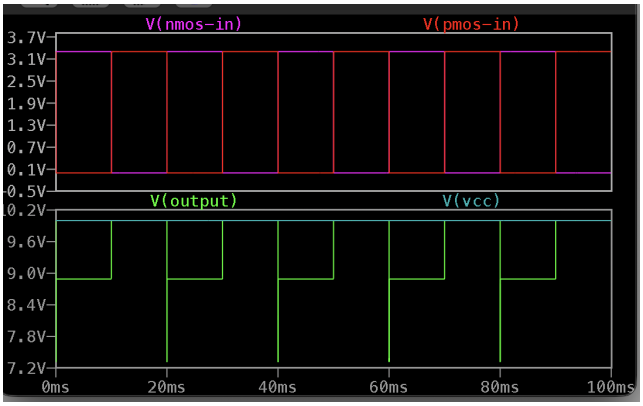


Fig. 9. High-voltage failure mode of the resistor-network half bridge.

This stage also made dead-time selection an explicit design choice. In the LTspice full-bridge simulation, the complementary commands were separated by 0.02 ms, or 20 µs. The local IRLZ14 and IRF9Z24 models include finite gate charge and capacitance values, with the IRLZ14 modeled at $Q_g = 8.4\,\mathrm{nC}$ and the IRF9Z24 modeled at $Q_g = 19\,\mathrm{nC}$. The BS170 helper stage and the gate resistor network determine how quickly those gates are charged and discharged, so the dead time must cover the interval where one device is still turning off before the other device turns on. A 20 µs blanking interval is conservative for the low-frequency prototype because it is much longer than the intrinsic MOSFET transition time represented by the model, but it is still only $0.1\%$ of the 20 ms square-wave command period. This makes the dead time large enough to reduce shoot-through risk in the model while keeping the visible output distortion limited.

## VII. Full-Bridge Extension

With the improved half bridge established, the full bridge followed directly. Two improved half bridges were placed side by side and driven with complementary commands. The main advantage was that the output was now measured between two switching nodes, giving a bipolar load voltage rather than a waveform that only moved between ground and the positive rail.

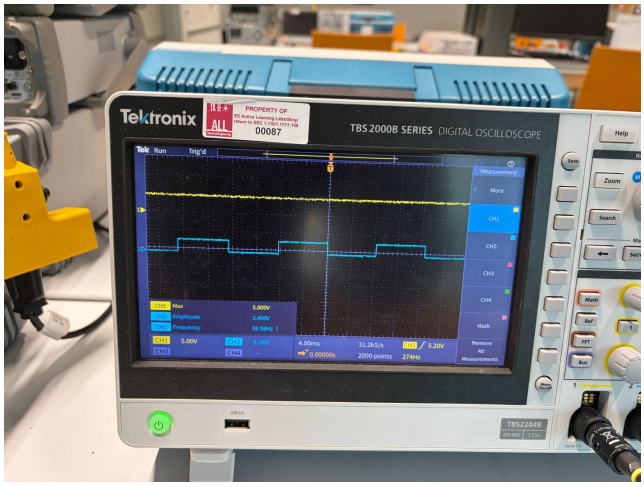


Fig. 10. Breadboarded resistor-network half bridge driven by the function generator. The output shape was recognizable, but the amplitude was lower than desired. Yellow: DC input. Blue: AC output

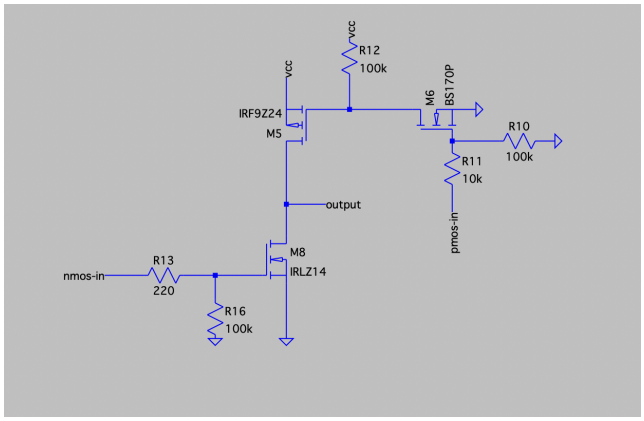


Fig. 11. Improved half-bridge design with BS170 helper transistor stage.

The full bridge was simulated with square-wave control and added dead time. The resulting output showed artefacts around switching edges, which is expected when dead time prevents the two devices in a bridge leg from switching instantaneously. This is one of the central tradeoffs in inverter design.

## VIII. SPWM Control

Square-wave operation proves that the bridge can switch power, but it is not the desired final waveform for AC systems. SPWM improves the output by comparing a sinusoidal reference to a high-frequency carrier waveform. The comparison produces a PWM gate command whose duty cycle varies sinusoidally over the output period.

$$v_{\text{ref}}(t) = M \sin(2\pi f_o t) \tag{1}$$

$$s(t) = \begin{cases} 1, & v_{\text{ref}}(t) > v_{\text{carrier}}(t) \\ 0, & v_{\text{ref}}(t) \leq v_{\text{carrier}}(t) \end{cases} \tag{2}$$

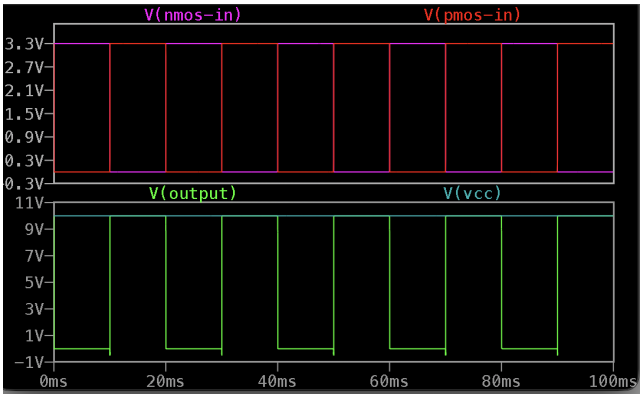


Fig. 12. Improved high-voltage behavior after adding the BS170 stage.

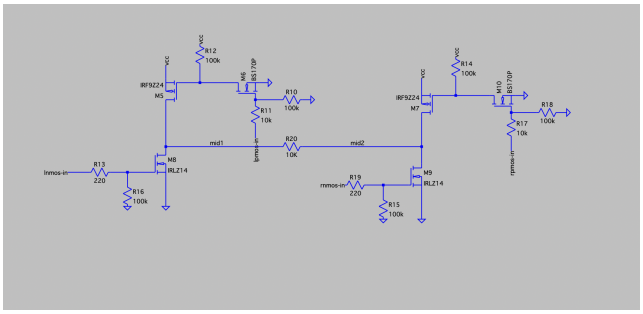

Fig. 13. Full bridge constructed from two improved half-bridge legs.

In LTspice, the SPWM control was implemented using behavioral voltage sources with conditional logic. Dead time was added to keep the simulated MOSFET bridge closer to a physically realizable circuit.

An RC low-pass filter was then added to attenuate the switching-frequency ripple and recover the low-frequency sinusoidal component as shown in Figure 17. This step is where the circuit begins to resemble a useful AC source rather than only a high-frequency switching network. This filter was further improved in Section XII-B

## IX. Three-Phase SPWM Inverter

The next extension was a three-phase inverter. Conceptually, this is three half bridges sharing the same DC link. Each half bridge drives one phase node, and the three phase commands are separated by 120°:

$$m_a(t) = M \sin(2\pi f_o t) \tag{3}$$

$$m_b(t) = M \sin(2\pi f_o t - 2\pi/3) \tag{4}$$

$$m_c(t) = M \sin(2\pi f_o t + 2\pi/3) \tag{5}$$

The STM32 generated three SPWM outputs using these phase-shifted references. In the data presented in Figure 18, the output frequency was approximately 10 Hz, the update rate was approximately 5 kHz, and the modulation index was set to approximately $M = 0.4$.

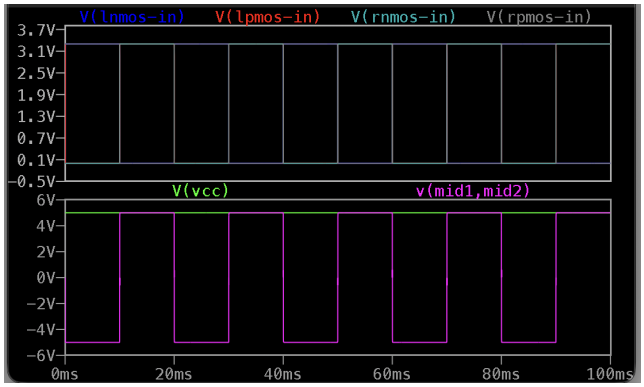


Fig. 14. Full-bridge square-wave output showing edge distortion associated with dead time.

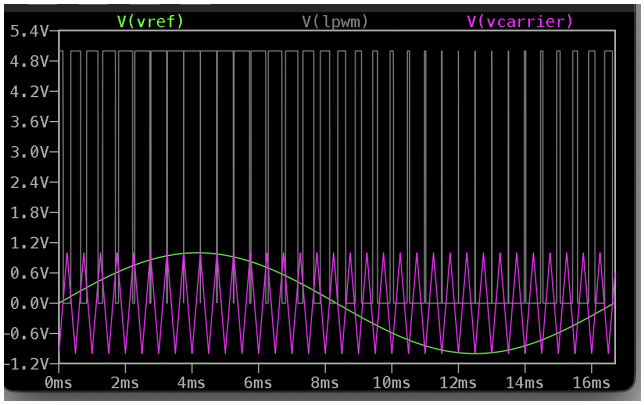


Fig. 15. SPWM: sinusoidal reference, carrier waveform, and resulting PWM command.

### A. STM32 Implementation

The STM32 Nucleo G474RE microcontroller board was used to implement control for the physical three-phase prototype [5]. STM32CubeMX was used to generate the base configuration (.ioc file), the VSCode STM32Cube extension was used for code development along with the CMake build toolchain. Code for this project is accessible at https://github.com/mark-rifkin/es145_fp.

The board includes a high-frequency built in clock (with maximum frequency at around 170 kHz), which was configured in STM's to drive accurate PWM frequency. The clock is used to generate SPWM duty cycles for the three channels. Each channel consists of a complementary output pair (e.g. CH1/CH1N). An advantage of the STM32 is that this complementarity is locked in hardware, along with dead-time to prevent shoothrough (the high and low transistors being enabled at the same time would cause a short circuit).

At each control update, the software determines the correct duty ratio to command based on a sinusoidal reference. The Nucleo can calculate trigonometric functions quickly, and so the references are generated live. The three sinusoidal references are offset by 120 degrees in code. are then used to calculate the duty ratio via the formula $d = \frac{1}{2}mv(t) + 1/2$, where $m$ is the modulation index (which scales down the output to avoid commanding more voltage than is possible). This formula is equivalent to the LTSpice-simulated method of comparing the signals to a carrier ranging from -1 to 1. [1].

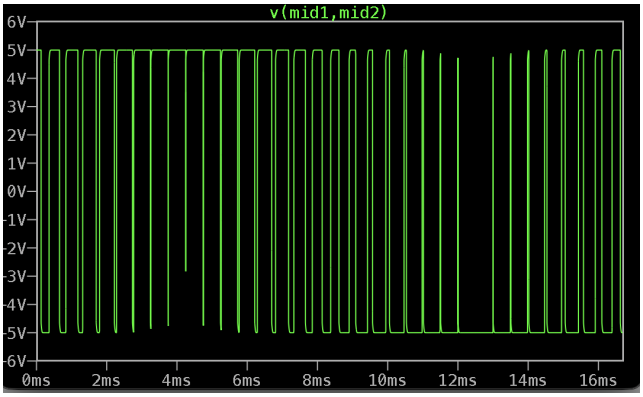


Fig. 16. Unfiltered full-bridge SPWM output in LTspice.

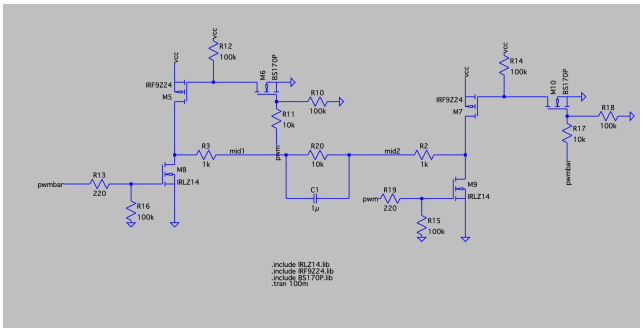

Fig. 17. RC low-pass filtering added to the full-bridge output.

## X. Breadboard Implementation and Filtering

The three-phase circuit was built on a breadboard and connected to a wye load. RC filters were added to the outputs so that the measured waveforms showed the lower-frequency sinusoidal components rather than only the switching waveform.

During testing, the switching action disturbed the supply rails enough that a capacitor had to be added between the positive and negative rails. This capacitor helped smooth the input rail and made the circuit more stable. This observation is consistent with practical inverter design as the DC link is typically not an ideal voltage source, and fast switching currents require local energy storage and decoupling.

## XI. Modulation Limits

The modulation index determines how strongly the sinusoidal reference uses the carrier range. At moderate modulation index, the three-phase output remained unclipped. As the modulation index increased, the reference began to overuse the available switching range and the output showed clipping. This is the transition toward overmodulation.

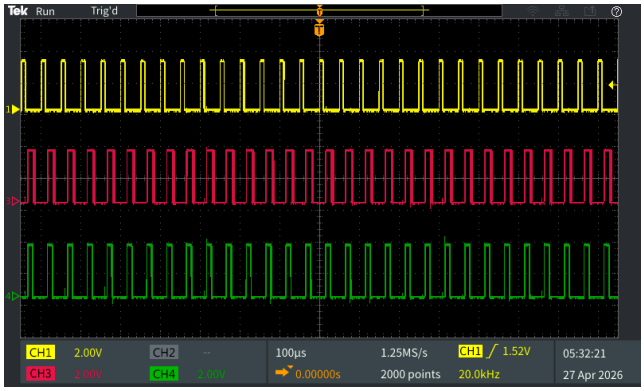


Fig. 18. STM32-generated three-phase SPWM outputs measured on the oscilloscope.

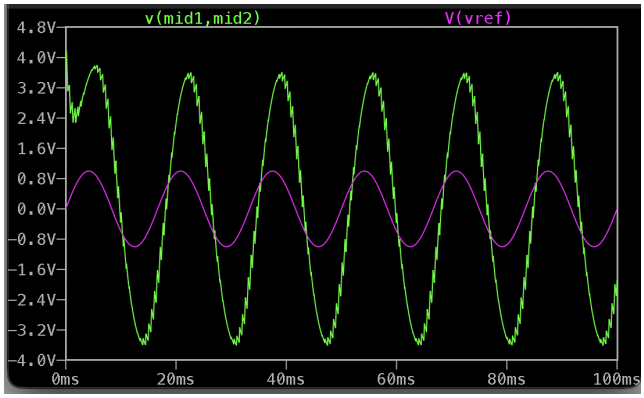


Fig. 19. Filtered full-bridge output compared with the sinusoidal reference.

## XII. Grid-following implementation

With the three-phase inverter implemented, the next step was acheiving simple tracking. Real-world inverters usually track the grid phase using a phase-locked loop, which takes the grid voltage as input and outputs the grid angle. This grid angle is then fed to a controller that sets the output current in order to achieve a desired real (aligned with grid) and reactive (perpendicular to grid) power injection. The following implementation is more basic: it determines the grid phase (and frequency) by measuring the times at which the grid angle crosses zero. The inverter voltage output is then set in phase with the grid voltage reference (so that it would be injecting only real power) [6].

For this section, the full-bridge circuit was used as it is simpler to build, debug, and measure than the three-phase circuit. Due to issues implementing this system on the STM32 Nucleo board (particularly, the toolchain stack being difficult to coordinate, and an unresolved gate driver problem), an Arduino Nano was used for control.

### A. Grid to ADC passive network

The ”grid voltage” is simulated by a sine wave at 50 Hz, with peak-to-peak amplitude 20 V in the final implementation (the benchtop function generator’s maximum voltage output).

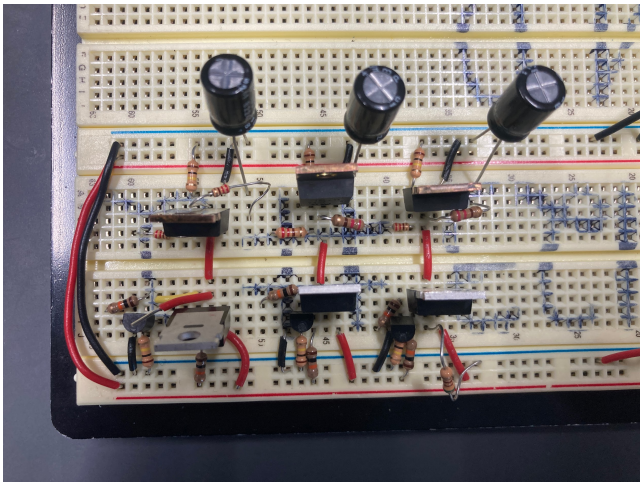

Fig. 20. Breadboard implementation of the three-phase inverter.

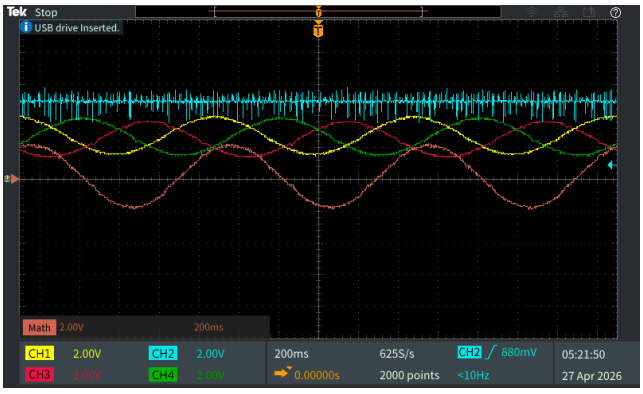


Fig. 21. Filtered three-phase output with a wye-connected load. Pink: line-to-line voltage. Yellow, green, red: phase-to-neutral voltages. Blue: DC input

The Arduino’s analog-digital converter (ADC) ports can sense analog voltages from 0-5 V. Thus, the circuit shown in Figure 25 was implemented to reduce the grid voltage peak-to-peak amplitude to $< 5$ V and center it at 2.5 V.

The leftmost part of the circuit is a voltage divider on the oscillating grid reference, with output $V_{out} = V_{in}\frac{R_2}{R_1+R_2}$. The resistors are configured to cut the peak-to-peak voltage to roughly 5 V. For grid voltage 20 V, the resistor values shown cut the peak-to-peak voltage to about 4.64 V. The right side is another voltage divider on a constant 5V reference (from the Arduino), with output voltage 2.5V. The capacitor is an AC coupler; acting like an open circuit to DC voltages but like a short to AC voltages. Thus, the voltage at ADC IN is the sum of 2.5V and the scaled AC grid reference.

### B. Full bridge and output filtering

The final voltage-following prototype used the same full-bridge logic from SectionVII, but replaced the fixed open-loop reference with a measured function-generator voltage reference. This let the bridge follow the phase and frequency of an external low-voltage “grid” signal. Although this was demonstrated on a single full bridge rather than on a complete

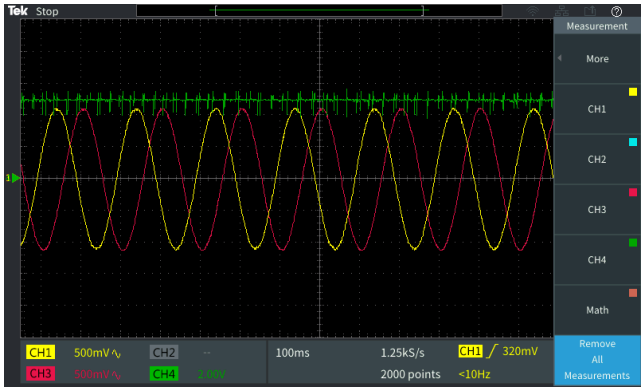


Fig. 22. Two selected phase outputs from the wye-connected load.

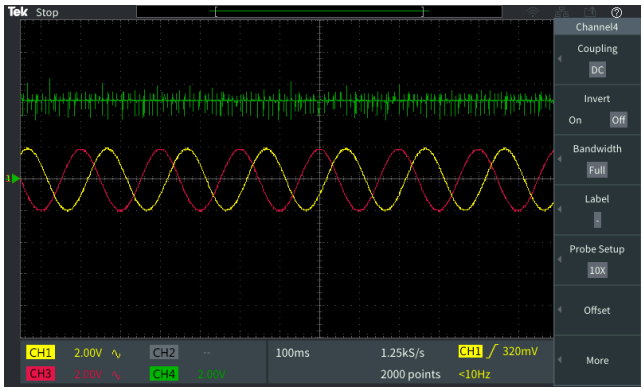


Fig. 23. Three-phase output at lower modulation without visible clipping.

three-phase follower, the result is directly extensible as a three-phase follower can be built by replicating the same bridge-leg structure and generating three SPWM references separated by \$120\^circ\$. In other words, the full bridge validated the sensing, synchronization, switching, and filtering decisions needed before scaling the same control idea to three phases.

In this final full-bridge test circuit, the filter uses a series resistance of $1\,\mathrm{k}\Omega$ on each side of the bridge output, along with the capacitor of $470\,\mathrm{nF}$ placed across the differential output.

The resulting differential resistance is therefore approximately

$$R_{\mathrm{eq}} \approx 2\,\mathrm{k}\Omega.$$

This gives a cutoff frequency of

$$f_c \approx \frac{1}{2\pi R_{\mathrm{eq}} C} = \frac{1}{2\pi \cdot (2,\mathrm{k}\Omega) \cdot (470,\mathrm{nF})} \approx 169\,\mathrm{Hz}. \quad (6)$$

This value is sufficiently high that the $60\,\mathrm{Hz}$ fundamental is not significantly attenuated, while being low enough to suppress the several-kilohertz PWM ripple present at the bridge output. See the final full bridge and the filter schematic in Figure 26 and the breadboard implementation in Figure 27.

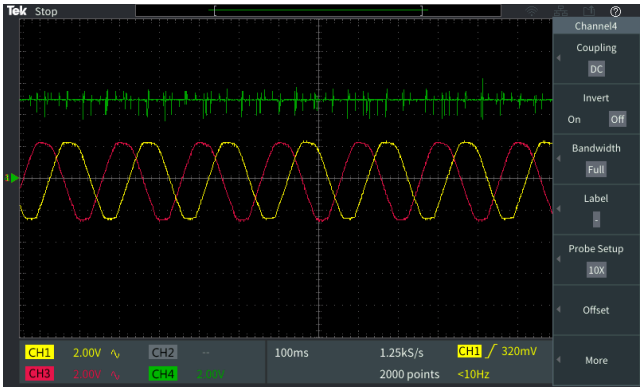


Fig. 24. Output clipping at higher modulation index.

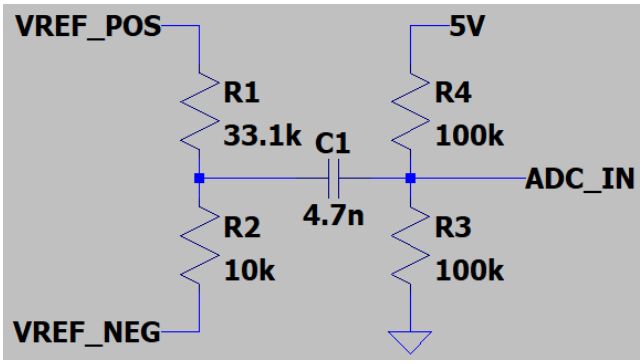


Fig. 25. ADC input passive circuit

### C. Arduino grid-following implementation

The grid-following full-bridge is implemented in Arduino C for the Nano controller. Unlike the STM32 board, the Nano cannot compute trigonometric functions in real-time, so a lookup table is computed in setup. For 256 points around a sine wave, the duty is computed as before for the STM32 and stored. Then, given a desired phase from 0-255 (converted from time based on the reference sine period) the controller looks up the correct duty cycle. The `outputOneCarrierPeriod` method generates a single period of the PWM output, including a high period, a low period, and deadtime. This is less reliable than the STM32 implementation, since it is software-based and not hardware-based, and thus the duty cycle and deadtime is not guaranteed. As visible in Figure 28, this results in the deadtime not being symmetrical, and limits the extent to which deadtime and modulation can be optimized on the Arduino without shoot-through or output compromises.

Nonetheless, the PWM generation on the Arduino functions very adequately to drive the full bridge. In fact, as seen in Figure 29, the Arduino implementation is able to handle a 20V DC input, whereas the STM32 implementation began to experience output degeneration starting at around a 8V DC input. This is an implementation error and not an inherent characteristic of the boards, since the STM32 is generally a more powerful board. The full bridge sees $\pm V_{DC}$ at any one

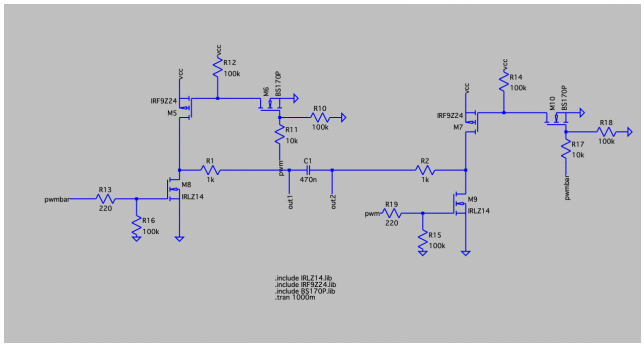


Fig. 26. Final full-bridge voltage-following LTspice schematic with the differential RC output filter.

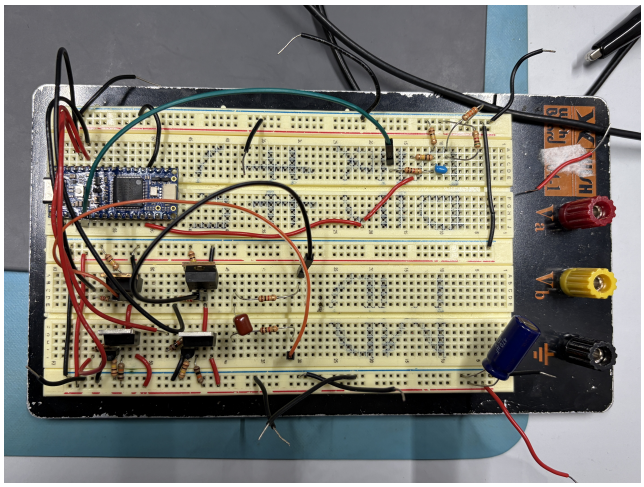

Fig. 27. Final Full Bridge implemented on a breadboard.

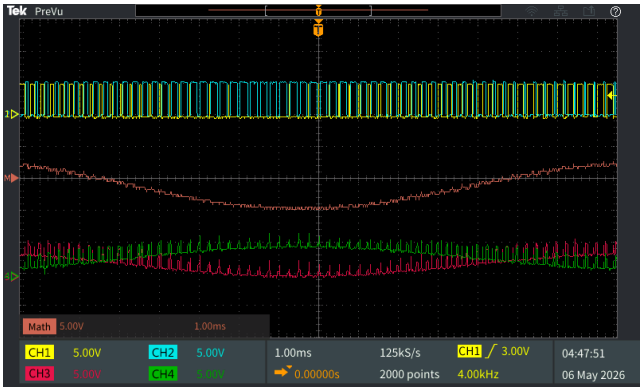


Fig. 28. Full-bridge PWM inputs and output (no grid reference). Blue and yellow: complementary PWM outputs. Pink: AC bridge output.

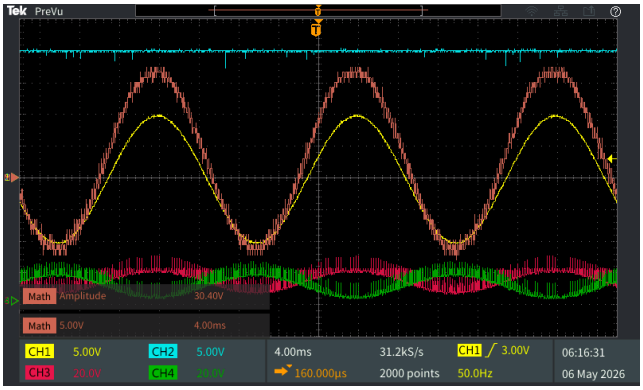


Fig. 29. Full-bridge PWM inputs and output, grid-tracking. Blue: DC input. Yellow: Grid reference. Pink: AC bridge output.

time, so the 20V input is converted into a 30V peak-to-peak AC output (given modulation index 0.7).

Figure 29 also demonstrates grid-following in action. The Arduino checks when the voltage on its ADC input crosses 0 (accounting for noise by using a hysteretic sensing system, where the threshold for upward approaches is slightly positive and the threshold for downward approaches is slightly negative), and calculates the period of its sinusoidal output from the difference between successive zero times. The phase can also be calculated from the zero times. This enables the generation of a phase-matched voltage output. The phase had to be manually offset, because there is some software lag between the phase being calculated from the grid reference and outputted to the transistor gates. If zero crossings are not detected on the ADC input, the controller falls back to a nominal frequency and arbitrary phase.

## XIII. Future Work

The voltage-following result establishes the synchronization side of grid-following control. The remaining step is current control, where the inverter must measure output current, compare it with a reference current in phase with the measured voltage, and adjust the modulation command to regulate injected real and reactive power. Future work should therefore implement current sensing on the output such as a Hall-effect sensor, a current transformer with appropriate burden and conditioning, or a shunt resistor with an isolated or differential current-sense amplifier.

A true phase-locked loop could also improve synchronization. The present zero-crossing method was sufficient for a clean function-generator reference, but a PLL would provide smoother phase estimates, better noise rejection, and a more natural path toward real grid synchronization. At the power stage level, a more robust implementation could use a dedicated gate-driver IC and possibly IGBTs or power MOSFETs selected specifically for inverter switching. Finally, once current-controlled grid-following operation is working, grid-forming control would be a natural next step because it would require the inverter to regulate voltage and frequency rather than only follow an existing voltage waveform.

## XIV. Conclusion

This project developed a low-voltage inverter prototype from ideal switching theory through MOSFET-level simulation, gate-drive iteration, full-bridge SPWM, output filtering, and voltage-following control. The ideal LTspice models made the half-bridge and full-bridge concepts clear, while

the MOSFET simulations and breadboard tests showed why real inverters require careful gate biasing, dead time, and switching-device selection. The BS170 helper stage improved the bridge drive behavior, and the final full-bridge circuit used complementary SPWM with a differential RC filter to produce a filtered AC output.

The final demonstration did not close a current loop, because we did not have a current sensor, and the transimpedance amplifier we hoped to modify into a current-sensing circuit did not produce a reliable feedback signal. However, the project did demonstrate all the major preceding steps, such as voltage sensing and phase/frequency following using a function generator as the grid reference. Since a three-phase inverter is built by replicating bridge legs and shifting the SPWM references by $120^\circ$, the working full-bridge voltage-following result provides a clear path to a future three-phase grid-following implementation.

## XV. Acknowledgements

We wish to thank Prof. Le Xie for a great introduction to this material and Qian Zhang for project and Problem Set feedback throughout the course. We also thank Leonardo Gomez from the EE Active Learning labs for support in acquiring the necessary hardware to make this project successful.

## References

[1] Imperix. *Three-phase Voltage Source Inverter (VSI)*. TN152. Posted March 30, 2021; updated January 19, 2026. Accessed 2026-05-05. Imperix. Jan. 19, 2026. URL: https://imperix.com/doc/implementation/three-phase-voltage-source-inverter.

[2] John G. Kassakian et al. *Principles of Power Electronics*. 2nd ed. Cambridge, UK: Cambridge University Press, 2024. ISBN: 9781316519516. DOI: 10.1017/9781009023894.

[3] Benjamin Kroposki and Andy Hoke. "A Path to 100 Percent Renewable Energy: Grid-Forming Inverters will Give Us the Grid We Need Now". In: *IEEE Spectrum* 61.5 (2024), pp. 50–57. DOI: 10.1109/MSPEC.2024.10523014.

[4] Yashen Lin et al. *Research Roadmap on Grid-Forming Inverters*. Technical Report NREL/TP-5D00-73476. Accessed 5 May 2026. National Renewable Energy Laboratory (NREL), Nov. 2020.

[5] STMicroelectronics. *UM2505: STM32G4 Nucleo-64 Boards (MB1367) — User Manual*. Accessed 5 May 2026. STMicroelectronics. June 2021.

[6] Alexander Tkachenko. "Parallel Operation of Grid-Forming Power Inverters". Available via DiVA portal, diva2:1751355. Master's thesis. Stockholm, Sweden: KTH Royal Institute of Technology, 2023. URL: https://kth.diva-portal.org/smash/get/diva2:1751355/FULLTEXT01.pdf.

[7] Wikipedia contributors. *Synchronization (alternating current)*. https://en.wikipedia.org/wiki/Synchronization_(alternating_current). Last edited 31 July 2024. Accessed 5 May 2026. 2024.